BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Allows access to and modification of caller variable

Q2. Where a variable is accessible

Q3. No value

Q4. Fast average access when well designed

You Do pages · Independent questions

Q1. Reuse and easier isolated testing/maintenance.

Q2. Pauses execution to inspect current state and follow iteration.

Q3. Replace Console.WriteLine with return a+b;

Exit Ticket · Model answer

True. Modular code separates responsibilities, making individual modules easier to test.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

RHS COMPUTING / OCR H446 / mixed LESSON 043 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

LESSON 043 · Programming Techniques · Modular Design and IDE Debugging

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What does ref do in C#?

Q2. What is variable scope?

Q3. What does a void method return?

Q4. State one advantage of a hash table.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — modularity

Recall the term — breakpoint

Recall the term — step into

Recall the term — watch

Recall the term — testability

New learning

Explain the important decision — ValidateInput() checks a value, CalculateTotal() computes the result and PrintReceipt() displays it. Each has one responsibility.

Worked example

Trace a method call

1. Identify the task — Trace a method call

2. Follow the logic — ValidateInput() checks a value, CalculateTotal() computes the result and PrintReceipt() displays it. Each has one responsibility.

3. Check the outcome — Explain how each step contributes to the result. Test an edge case.

We Do · Practise with guidance

Problem 1

A method has no return despite return type int. What is wrong?

Discuss, then respond

Check your thinking

Not all code paths return a value.

Problem 2

Why step into a function in a debugger?

To inspect statements inside the called method.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Apply independently
Give two benefits of modular design. [2]

Question 2

Apply independently
Describe how a breakpoint helps debug a loop. [2]

Question 3

Apply independently
Correct: static int Add(int a,int b){ Console.WriteLine(a+b); } [2]

Exit Ticket · Answer before checking

True or false: modular code can be easier to test independently.